Midterm Overview

Database Design and Web Implementation



Midterm Logistics



* **Held During class on Tuesday, March 27th**
* **11:00am - 12:15pm**
* **You will use Brightspace and Respondus (the practice exam will prompt you to download Respondus if you don’t already have it installed)**
* **If you need an accommodation you must request it directly from the Moses Center**

Midterm Exam Structure

There will be no multiple choice questions. All questions will require short responses.

Two sections:

Section 1

10 Questions:

Python Lists and Dictionaries
Normalization
PK's and FK's
Relationships
DDL and DML Commands practiced in class

Section 2

5 Questions:

Modelling
Tables and Attributes
PK's and FK's
Relationships
Datatypes
Select Statements

Section 1

11 short answer questions.

Python Lists and Dictionaries
Normalization
PK's and FK's
Relationships
DDL and DML covered in class

Sample Python Lists/Dictionaries Question

Explain hashable in the context of the structure below. Which parts are hashable and why?

```
students = {
    101: {"name": "Ava", "major": "CS"},
    102: {"name": "Liam", "major": "Math"},
    103: {"name": "Noah", "major": "CS"}
}
```

Structurally, this is a dictionary whose values are dictionary objects. The keys (101, 102, 103)(“name”,”major”) are hashable since their values cannot change after creation. They are immutable so their value remains consistent.
Python uses the hash values of dictionary keys to store and retrieve entries efficiently.
The dictionary objects themselves are not hashable because they are mutable.

Sample PK/FK Question

Based solely on the information provided below, which field or fields are primary keys and which are foreign keys?

customer

customerId	customerName
1	John Smith
2	Harry Jones
3	Ariel Brown

account

accountId	customerId	accountType	dateOpened
2302	1	Checking	1/1/2024
3759	2	Savings	1/15/2024
4334	2	Checking	1/15/2024

A primary key uniquely identifies each row. In the customer table customerId is the primary key. In the account table accountId is the primary key.

account.customerId is a foreign key that references customer.customerId.

Sample Relationship Question

The following diagram shows two tables that are part of a school database. What is the cardinality of the relationship between these two tables? If it is one to many, indicate which is the one side and which the many side. If there is anything missing from these tables that is required to implement the relationship, please indicate what that is.

student

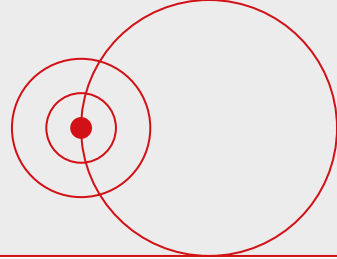
studentId	studentName
1	John Smith
2	Harry Jones
3	Ariel Brown

course

courseId	courseName
2302	1
3759	2
4334	2

this is a many to many relationship, a student can take many courses, a course can have many students. Since SQL cannot directly implement many to many relationships we would need a bridging table to implement this. The bridging table could be called something like enrollment and would use a composite primary key composed of studentId and courseId and could hold information unique to enrollment.

Sample Normalization Question

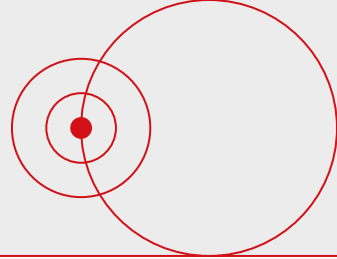


Which Normal Form is the following table in? Why is it not in the next higher Normal Form? (You do not get credit for answering the first part correctly if you do not answer the second part correctly)

rental						
customerId	carId	carColor	carType	dateOut	datein	agentId
34432	432	Red	Sedan	9/1	9/2	43
36574	354	Red	SUV	9/3	9/10	23
35432	432	Red	Sedan	9/5	9/6	34
36475	446	Black	Coupe	9/6	9/15	23
36465	324	White	Sedan	9/7	9/8	32

This table is in 1NF because all data is atomic, there are no repeating columns. It's not in 2NF because customerId and carId are needed to create a unique composite primary key and values like carColor or carType would only be dependent on carId. This is a partial dependency and violates 2NF. To get it into 2NF we'd have to separate this table into a car table and customer table.

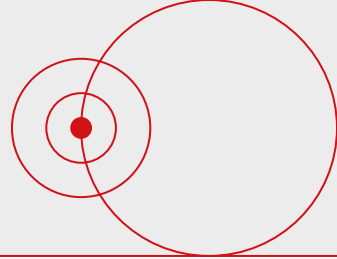
Sample DDL Question



What is the SQL statement that will delete the student table?

DROP TABLE student;

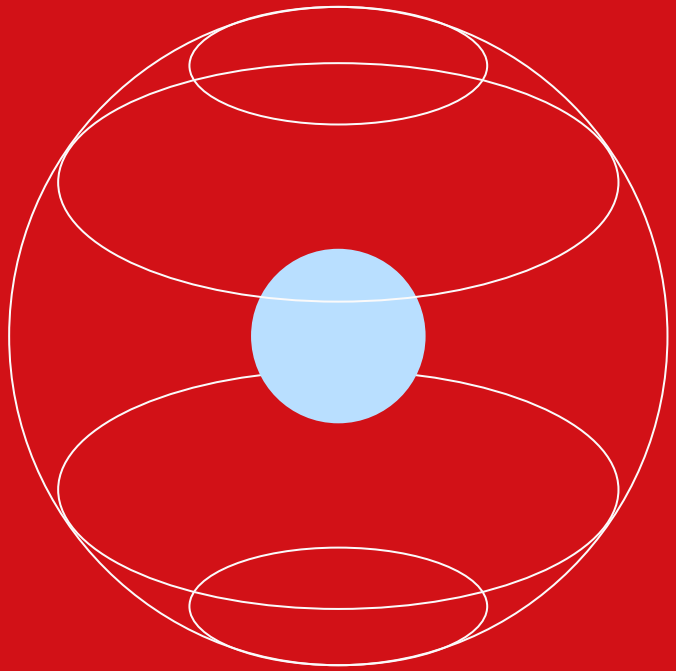
Sample DML Question



What is the SQL statement that will delete the record for “Jimmy Brown” (value stored in the column customerName) from the customer table?

DELETE FROM customer WHERE customerName = “Jimmy Brown”;

Section 2



Contains a short entity/organization description.
The 4 questions relate to the tables and relationships that model the description.
The questions will cover all aspects of modelling, including:
Tables and attributes
PKs/FKs
Relationships
Datatypes
***Basic Select Statements**

Sample Scenario

- An animal shelter

You have been asked to design a database for an animal/pet shelter.

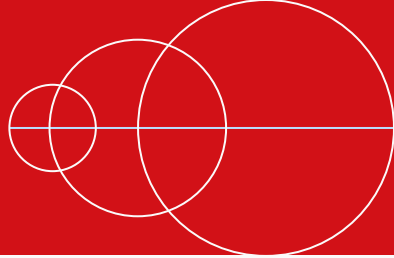
The shelter keeps track of the animals currently housed there, the people who adopt them, and the employees who manage their care.

Each pet has an ID number, a name, a species (dog, cat, etc.), a breed, a date of arrival, and an employee responsible for its care.

Each employee has an ID number, a name, a position (e.g., manager, vet tech, volunteer), and a hire date.
Each adopter has an ID number, a name, a phone number, and an email address.

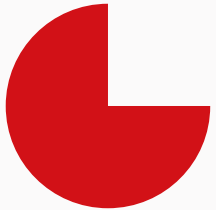
When an adopter adopts a pet, the shelter records the date of adoption and which employee handled the adoption paperwork.

In sqlite create a database that addresses these questions: (you will upload your .db and .sql files to brightspace along with typing in text). You can use lucid app to sketch an ERD if it is helpful.



1. *Which tables are needed?*
2. *What are the primary and foreign keys?*
3. *What are the relationship types and cardinalities*
4. *What SQLite data types would you choose for each attribute?*
5. *Write a simple SELECT query:*

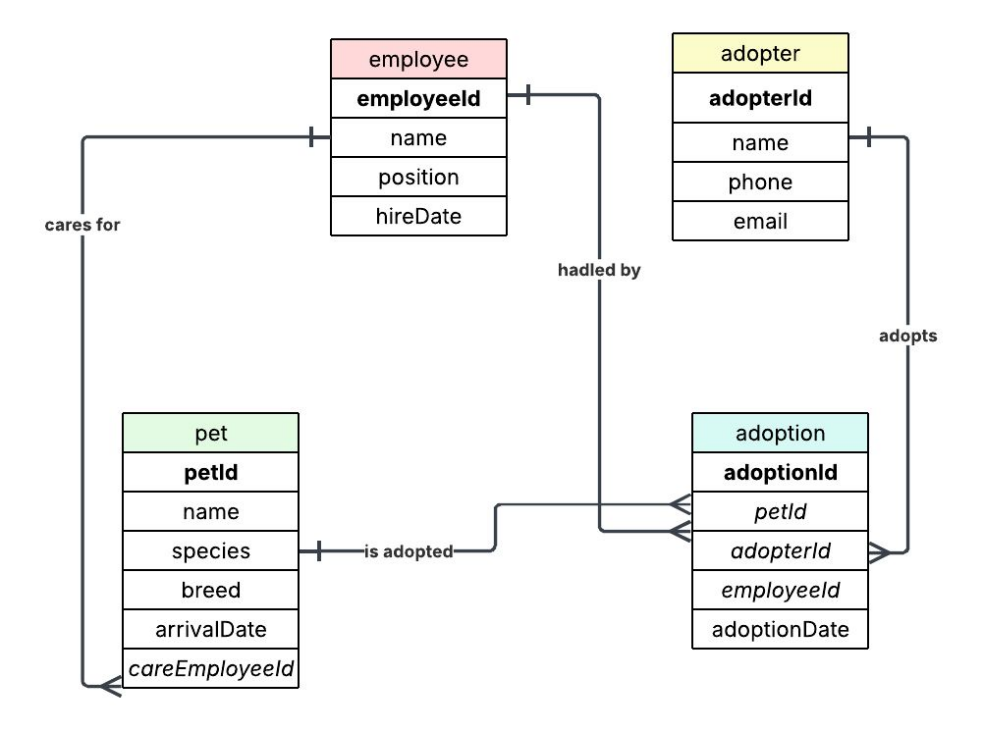
“Write an SQL statement that shows the name of each adopted pet, the adopter’s name, and the adoption date.”



1. Which tables are needed?

- employee — staff/volunteers who care for pets and handle adoptions
- adopter — people who adopt pets
- pet — animals currently/previously housed
- adoption — event/transaction table that records an adoption (date + employee who handled paperwork)

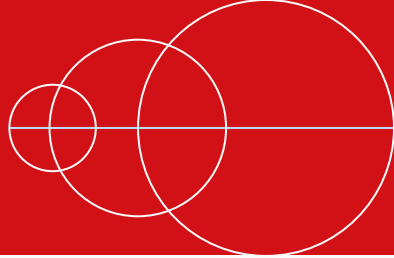
Example ERD


employee
employeeId
name
position
hireDate
adopter
adopterId
name
phone
email
cares for
hadled by
adopts
pet
petId
name
species
breed
arrivalDate
careEmployeeId
is adopted
adoption
adoptionId
petId
adopterId
employeeId
adoptionDate

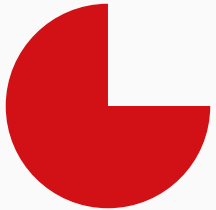
2. What are the primary and foreign keys?

- employee(employeeId) primary key
- adopter(adopterId) primary key
- pet(petId) primary key
- adoption(adoptionId) primary key
- adoption.petId → FK to pet(petId) (plus UNIQUE so a pet is adopted at most once)
- adoption.adopterId → FK to adopter(adopterId)
- adoption.employeeId → FK to employee(employeeId) (paperwork handler)
- pet.careEmployeeId → FK to employee(employeeId) (care assignment)

3. What are the relationship types and cardinalities



- *employee (1) → (many) pet via pet.careEmployeeId*
- *one employee can be responsible for many pets; each pet has one responsible employee (assumption per scenario).*
- *adopter (1) → (many) adoption*
- *one adopter can adopt many pets over time.*
- *employee (1) → (many) adoption*
- *one employee can handle many adoptions.*
- *pet (1) → (0 or 1) adoption*
- *a pet may be unadopted (still in shelter) or adopted once. Enforced by UNIQUE(adoption.petId).*



4. What SQLite data types would you choose for each attribute?

- *IDs: INTEGER (use as PK)*
- *Names/species/breed/position/phone/email: TEXT*
- *Dates: TEXT in YYYY-MM-DD format (common SQLite convention)*

- *Constraints: NOT NULL, CHECK(...), UNIQUE(...) where appropriate*

4. DDL to create database with SQLite

```
PRAGMA foreign_keys = ON;

CREATE TABLE employee (
  employeeId   INTEGER PRIMARY KEY,
  name         TEXT NOT NULL,
  position     TEXT NOT NULL,
  hireDate     TEXT NOT NULL  -- YYYY-MM-DD
);

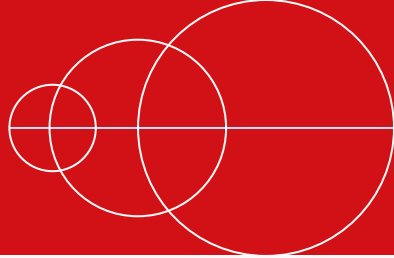
CREATE TABLE adopter (
  adopterId    INTEGER PRIMARY KEY,
  name         TEXT NOT NULL,
  phone        TEXT,
  email        TEXT UNIQUE
);

CREATE TABLE pet (
  petId           INTEGER PRIMARY KEY,
  name            TEXT NOT NULL,
  species         TEXT NOT NULL,        -- e.g., dog, cat
  breed           TEXT,
  arrivalDate     TEXT NOT NULL,        -- YYYY-MM-DD
  careEmployeeId  INTEGER NOT NULL,
  FOREIGN KEY (careEmployeeId) REFERENCES employee(employeeId)
);

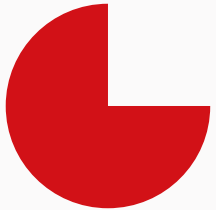
CREATE TABLE adoption (
  adoptionId   INTEGER PRIMARY KEY,
  petId        INTEGER NOT NULL UNIQUE,
  adopterId    INTEGER NOT NULL,
  employeeId   INTEGER NOT NULL,         -- handled paperwork
  adoptionDate TEXT NOT NULL,            -- YYYY-MM-DD

  FOREIGN KEY (petId)     REFERENCES pet(petId),
  FOREIGN KEY (adopterId) REFERENCES adopter(adopterId),
  FOREIGN KEY (employeeId) REFERENCES employee(employeeId)
);
```

5. Write a simple SELECT query:
“Write an SQL statement that shows the name of each adopted pet, the adopter’s name, and the adoption date.”

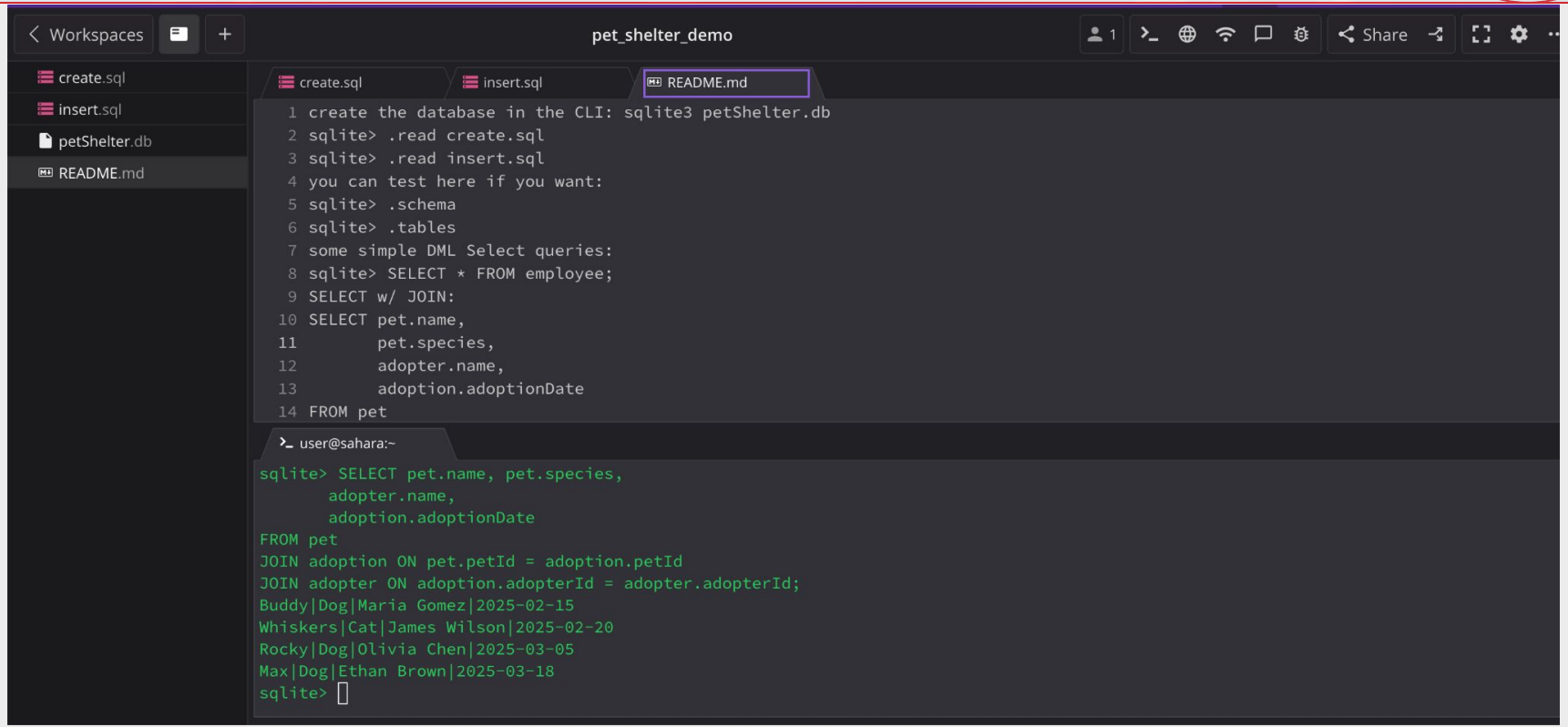


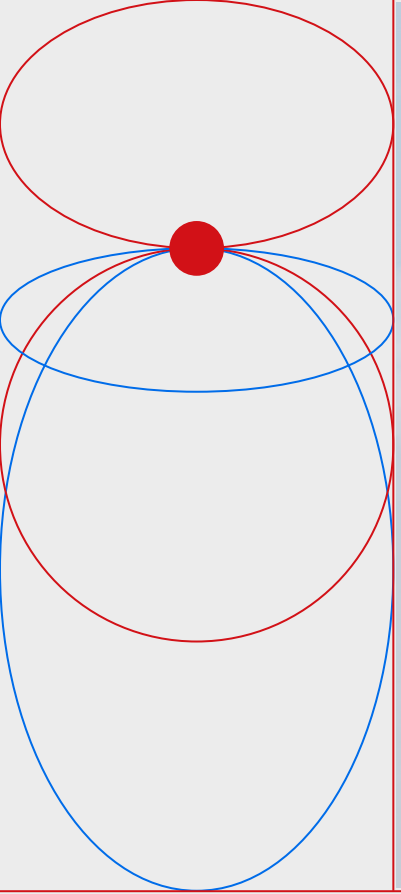
```
SELECT pet.name,
    pet.species,
    adopter.name,
    adoption.adoptionDate
FROM pet
JOIN adoption ON pet.petId = adoption.petId
JOIN adopter ON adoption.adopterId = adopter.adopterId;
```



Example edstem repo:

https://edstem.org/us/courses/105050/workspaces/phLbag2U97hKOQ8raxFBZ4WKpG0RQa3i

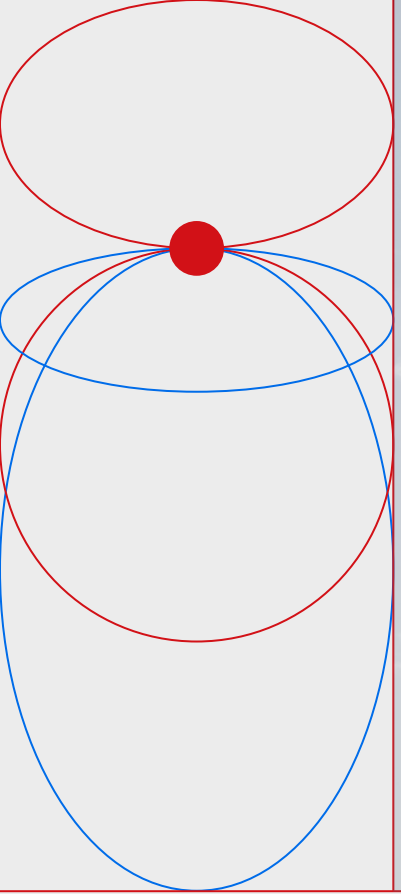




What This Exam Is Testing

- *Do you understand how relational databases are structured?*
- *Can you translate a description into a schema?*
- *Can you implement that schema in SQL?*
- *Can you write a simple query that reflects the relationships?*

It is not testing memorization of obscure syntax and acronyms, principles of relational algebra or set theory.



Study tips:

- **Model a scenario**

Be able to identify entities, primary keys, foreign keys, and relationship types from a short description.

- **Write CREATE TABLE statements**

Practice declaring primary keys, foreign keys, NOT NULL, UNIQUE, and appropriate SQLite data types.

- **Explain relationships clearly**

Be able to describe one-to-many and many-to-many relationships and when a bridging table is required.

- **Understand normalization**

Identify partial dependencies and explain why a table is not in 2NF and how to fix it.

- **Write JOIN queries**

Practice writing 2-table and 3-table JOINs using foreign keys correctly.

- **Focus on structure, not formatting**

You are graded on logical correctness and reasoning, not minor syntax details.

- **Understand python list, dictionary, and tuple types as data structures and perform simple for loops and slicing.**